

TP4 - Authentification Complète et Stockage Sécurisé

Objectifs

- Implémenter un système d'authentification complet
- Utiliser le stockage sécurisé pour les tokens
- Gérer la navigation conditionnelle
- Créer un contexte d'authentification

Prérequis

- TP1, TP2 et TP3 terminés
- React Navigation installé

Étape 1 : Installation des dépendances

Stockage sécurisé

expo install expo-secure-store

Client HTTP pour les appels API

npm install axios

Gestion des tokens JWT (optionnel)

npm install jwt-decode

Étape 2 : Créer le contexte d'authentification

Créer le fichier AuthContext.js

```
import React, { createContext, useState, useContext, useEffect } from 'react';
```

```
import * as SecureStore from 'expo-secure-store';
```

```
import axios from 'axios';
```

```
// Créer le contexte
```

```
const AuthContext = createContext({});
```

```
// URL de base de votre API (à modifier)
```

```
const API_URL = 'https://votre-api.com/api';
```

```
// Hook personnalisé pour utiliser le contexte
export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error('useAuth doit être utilisé dans AuthProvider');
  }
  return context;
};
```

```
// Provider du contexte
export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [isLoading, setIsLoading] = useState(true);
  const [authToken, setAuthToken] = useState(null);
```

```
// Vérifier si l'utilisateur est déjà connecté au démarrage
useEffect(() => {
  checkAuthState();
}, []);
```

```
// Configurer axios avec le token
useEffect(() => {
  if (authToken) {
    axios.defaults.headers.common['Authorization'] = `Bearer ${authToken}`;
  } else {
    delete axios.defaults.headers.common['Authorization'];
  }
}, [authToken]);
```

```

// Vérifier l'état de l'authentification
const checkAuthState = async () => {
  try {
    const token = await SecureStore.getItemAsync('authToken');
    const userData = await SecureStore.getItemAsync('userData');

    if (token && userData) {
      setAuthToken(token);
      setUser(JSON.parse(userData));

      // Optionnel : Vérifier si le token est encore valide
      // await verifyToken(token);
    }
  } catch (error) {
    console.error('Erreur lors de la vérification de l\'auth:', error);
  } finally {
    setIsLoading(false);
  }
};

```

```

// Fonction de connexion
const login = async (email, password) => {
  try {
    // Appel API pour la connexion
    const response = await axios.post(`${API_URL}/auth/login`, {
      email,
      password
    });
  }
};

```

```

const { token, user } = response.data;

// Sauvegarder le token et les données utilisateur
await SecureStore.setItemAsync('authToken', token);
await SecureStore.setItemAsync('userData', JSON.stringify(user));

// Mettre à jour l'état
setAuthToken(token);
setUser(user);

return { success: true };
} catch (error) {
  console.error('Erreur de connexion:', error);
  return {
    success: false,
    error: error.response?.data?.message || 'Erreur de connexion'
  };
}
};

// Fonction d'inscription
const register = async (userData) => {
  try {
    // Appel API pour l'inscription
    const response = await axios.post(`${API_URL}/auth/register`, userData);

    const { token, user } = response.data;

```

```
// Sauvegarder automatiquement après inscription
await SecureStore.setItemAsync('authToken', token);
await SecureStore.setItemAsync('userData', JSON.stringify(user));

setAuthToken(token);
setUser(user);

return { success: true };
} catch (error) {
  console.error('Erreur d\'inscription:', error);
  return {
    success: false,
    error: error.response?.data?.message || 'Erreur d\'inscription'
  };
}
};
```

```
// Fonction de déconnexion
const logout = async () => {
  try {
    // Supprimer les données stockées
    await SecureStore.deleteItemAsync('authToken');
    await SecureStore.deleteItemAsync('userData');

    // Réinitialiser l'état
    setAuthToken(null);
    setUser(null);

    // Optionnel : Appeler l'API pour invalider le token
```

```

    // await axios.post(`${API_URL}/auth/logout`);
  } catch (error) {
    console.error('Erreur lors de la déconnexion:', error);
  }
};

// Mettre à jour le profil utilisateur
const updateProfile = async (updatedData) => {
  try {
    const response = await axios.put(`${API_URL}/user/profile`, updatedData);

    const updatedUser = response.data;
    setUser(updatedUser);

    // Mettre à jour le stockage
    await SecureStore.setItemAsync('userData', JSON.stringify(updatedUser));

    return { success: true };
  } catch (error) {
    console.error('Erreur de mise à jour:', error);
    return {
      success: false,
      error: error.response?.data?.message || 'Erreur de mise à jour'
    };
  }
};

const value = {
  user,

```

```

    isLoading,
    isAuthenticated: !!user,
    login,
    register,
    logout,
    updateProfile,
  };

  return (
    <AuthContext.Provider value={value}>
      {children}
    </AuthContext.Provider>
  );
};

```

Étape 3 : Modifier App.js pour utiliser le contexte

```

import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createStackNavigator } from '@react-navigation/stack';
import { AuthProvider, useAuth } from './AuthContext';
import LoginScreen from './LoginScreen';
import RegisterScreen from './RegisterScreen';
import HomeScreen from './HomeScreen';
import SplashScreen from './SplashScreen';

const Stack = createStackNavigator();

// Navigation principale
function AppNavigator() {
  const { isAuthenticated, isLoading } = useAuth();

```

```

if (isLoading) {
  // Afficher un écran de chargement
  return <SplashScreen />;
}

return (
  <Stack.Navigator>
    {isAuthenticated ? (
      // Écrans pour utilisateurs connectés
      <>
        <Stack.Screen
          name="Home"
          component={HomeScreen}
          options={{
            title: 'Accueil',
            headerRight: () => <LogoutButton />
          }}
        />
        { /* Ajouter d'autres écrans protégés ici */ }
      </>
    ) : (
      // Écrans pour utilisateurs non connectés
      <>
        <Stack.Screen
          name="Login"
          component={LoginScreen}
          options={{ title: 'Connexion' }}
        />

```

```

        <Stack.Screen
          name="Register"
          component={RegisterScreen}
          options={{ title: 'Inscription' }}
        />
      </>
    )}
  </Stack.Navigator>
);
}

```

```

// App principale
export default function App() {
  return (
    <AuthProvider>
      <NavigationContainer>
        <AppNavigator />
      </NavigationContainer>
    </AuthProvider>
  );
}

```

Étape 4 : Créer l'écran de chargement

Fichier SplashScreen.js

```

import React from 'react';
import { View, ActivityIndicator, StyleSheet } from 'react-native';

export default function SplashScreen() {
  return (
    <View style={styles.container}>

```

```

        <ActivityIndicator size="large" color="#007AFF" />
    </View>
);
}

```

```

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#f5f5f5',
  },
});

```

Étape 5 : Modifier LoginScreen pour utiliser le contexte

```

import React, { useState } from 'react';
import { /* imports existants */ } from 'react-native';
import { useAuth } from '../AuthContext';

export default function LoginScreen({ navigation }) {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [isLoading, setIsLoading] = useState(false);
  const { login } = useAuth();

  const handleLogin = async () => {
    if (!email || !password) {
      alert('Veuillez remplir tous les champs');
      return;
    }
  }
}

```

```
setIsLoading(true);

const result = await login(email, password);

setIsLoading(false);

if (!result.success) {
  alert(result.error);
}

// Si succès, la navigation se fait automatiquement
};

// Le reste du composant reste identique
// ...
}
```

Étape 6 : Créer l'écran d'accueil

Fichier HomeScreen.js

```
import React from 'react';

import {
  View,
  Text,
  StyleSheet,
  TouchableOpacity,
  ScrollView
} from 'react-native';

import { useAuth } from '../AuthContext';

export default function HomeScreen() {
  const { user, logout } = useAuth();
```

```

return (
  <ScrollView style={styles.container}>
    <View style={styles.header}>
      <Text style={styles.welcomeText}>
        Bienvenue, {user?.firstName} !
      </Text>
      <Text style={styles.emailText}>{user?.email}</Text>
    </View>

    <View style={styles.card}>
      <Text style={styles.cardTitle}>Informations du profil</Text>
      <View style={styles.infoRow}>
        <Text style={styles.label}>Nom complet:</Text>
        <Text style={styles.value}>
          {user?.firstName} {user?.lastName}
        </Text>
      </View>
      <View style={styles.infoRow}>
        <Text style={styles.label}>Téléphone:</Text>
        <Text style={styles.value}>{user?.phone || 'Non renseigné'}</Text>
      </View>
    </View>

    <TouchableOpacity style={styles.logoutButton} onPress={logout}>
      <Text style={styles.logoutText}>Se déconnecter</Text>
    </TouchableOpacity>
  </ScrollView>
);
}

```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f5f5f5',
  },
  header: {
    backgroundColor: '#007AFF',
    padding: 20,
    paddingTop: 40,
  },
  welcomeText: {
    fontSize: 24,
    fontWeight: 'bold',
    color: 'white',
    marginBottom: 5,
  },
  emailText: {
    fontSize: 16,
    color: 'rgba(255,255,255,0.8)',
  },
  card: {
    backgroundColor: 'white',
    margin: 20,
    padding: 20,
    borderRadius: 10,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.1,
```

```
    shadowRadius: 4,  
    elevation: 5,  
  },  
  cardTitle: {  
    fontSize: 18,  
    fontWeight: 'bold',  
    marginBottom: 15,  
    color: '#333',  
  },  
  infoRow: {  
    flexDirection: 'row',  
    justifyContent: 'space-between',  
    marginBottom: 10,  
  },  
  label: {  
    fontSize: 16,  
    color: '#666',  
  },  
  value: {  
    fontSize: 16,  
    color: '#333',  
    fontWeight: '500',  
  },  
  logoutButton: {  
    backgroundColor: '#ff3b30',  
    margin: 20,  
    padding: 15,  
    borderRadius: 8,  
    alignItems: 'center',
```

```
    },  
    logoutText: {  
      color: 'white',  
      fontSize: 16,  
      fontWeight: 'bold',  
    },  
  });
```

Étape 7 : Gérer les erreurs et la reconnexion

Intercepteur Axios pour gérer l'expiration du token

// Dans AuthContext.js, ajouter après la configuration axios

```
// Intercepteur de réponse  
axios.interceptors.response.use(  
  (response) => response,  
  async (error) => {  
    const originalRequest = error.config;  
  
    if (error.response?.status === 401 && !originalRequest._retry) {  
      originalRequest._retry = true;  
  
      // Token expiré, déconnecter l'utilisateur  
      await logout();  
  
      // Optionnel : Rediriger vers login  
      // navigation.navigate('Login');  
    }  
  
    return Promise.reject(error);  
  }  
);
```

```
);
```

Étape 8 : Ajouter la persistance de session

Refresh Token (optionnel)

```
// Dans AuthContext.js
```

```
const refreshAuthToken = async () => {  
  try {  
    const refreshToken = await SecureStore.getItemAsync('refreshToken');  
  
    if (refreshToken) {  
      const response = await axios.post(`${API_URL}/auth/refresh`, {  
        refreshToken  
      });  
  
      const { token } = response.data;  
      await SecureStore.setItemAsync('authToken', token);  
      setAuthToken(token);  
  
      return token;  
    }  
  } catch (error) {  
    console.error('Erreur de refresh:', error);  
    await logout();  
  }  
};
```

Bonnes pratiques de sécurité

1. Validation des certificats SSL

```
// Pour la production uniquement
```

```
axios.defaults.validateStatus = (status) => {  
  return status >= 200 && status < 300;  
};
```

```
};
```

2. Timeout des requêtes

```
axios.defaults.timeout = 10000; // 10 secondes
```

3. Chiffrement des données sensibles

```
// Utiliser expo-crypto pour chiffrer des données supplémentaires
```

```
import * as Crypto from 'expo-crypto';
```

```
const hashPassword = async (password) => {  
  const digest = await Crypto.digestStringAsync(  
    Crypto.CryptoDigestAlgorithm.SHA256,  
    password  
  );  
  return digest;  
};
```

Exercices pratiques

1. Ajouter la biométrie

- Utiliser expo-local-authentication
- Permettre la connexion par empreinte/Face ID

2. Implémenter "Se souvenir de moi"

- Checkbox sur l'écran de connexion
- Gérer la durée de vie du token

3. Ajouter la réinitialisation du mot de passe

- Écran "Mot de passe oublié"
- Envoi d'email de réinitialisation

4. Créer un écran de profil

- Afficher les informations utilisateur
- Permettre la modification

Gestion des états de l'application

```
import { AppState } from 'react-native';
```

```
// Dans AuthContext

useEffect(() => {

  const subscription = AppState.addListener('change', (nextAppState) => {

    if (nextAppState === 'active') {

      // L'app revient au premier plan

      checkAuthState();

    }

  });

  return () => subscription.remove();

}, []);
```

Dépannage

"Network request failed"

- Vérifier l'URL de l'API
- Vérifier la connexion internet
- Pour localhost, utiliser l'IP de la machine

Token non persistant

- Vérifier les permissions de l'app
- Tester sur un vrai appareil